DAY 4

Multi-Agent Systems & Human-in-the-Loop

From single agents to collaborative, safe AI systems

What We'll Cover Today

KAUST Academy

Five sections, from agentic retrieval to reliability

Yesterday: one agent, one loop. **Today:** retrieval as a decision, then teams.

TODAY'S LABS **Agent Team · Agentic RAG**

PART 01

From RAG to Agentic RAG

The agent assumes control of retrieval — deciding when, whether and how to search.

Query analysis · Self-correction · Multi-step retrieval

PART 02

Multi-Agent Orchestration

Patterns for coordinating specialist agents rather than overloading one.

Router · Supervisor–worker · Peer · Hybrid

PART 03

Agent Communication & State

How agents exchange information without losing context.

Shared state · Structured messages · Cost management

PART 04

Human-in-the-Loop

Retaining human control over risky, irreversible actions.

Trust spectrum · Approval gates · Escalation

PART 05

Failure Modes & Reliability

How multi-agent systems fail — and how to make them robust.

Cascading errors · Loop detection · Mitigation

OBJECTIVE **By the close of the session you will be able to design, coordinate and safeguard a team of agents — and apply it in the capstone project.**

PART 1

From RAG to Agentic RAG

What happens when the agent takes control of retrieval

Classic RAG: Four Limitations

One fixed path cannot fit every question

Always Retrieves

Searches the vector DB on every query — even "What's 2+2?" Wastes time and money, injects irrelevant context.

Fixed Strategy

Same retrieval approach every time. Can't adapt between keyword, semantic, or different data sources.

No Refinement

If the first retrieval fails, it doesn't retry — it just passes whatever it got to the LLM.

Day 2 lab: retrieval missed the $120/day line, and the model confidently answered $80.

Single-Shot Only

Can't handle questions needing multiple searches, like "Compare Q2 and Q3 performance."

What Is Agentic RAG?

KAUST Academy

Retrieval becomes a choice, not a step



Agentic RAG = an AI agent that decides **when, whether, how, and how many times** to retrieve information.

Day 2 listed Self-RAG / CRAG as "for another day." **Today is that day.**

FOUR DECISIONS THE AGENT OWNS

WHEN

When to retrieve

before answering, or mid-reasoning?

WHETHER

Whether to retrieve

skip it entirely for simple queries

HOW

How to retrieve

semantic, keyword, SQL, or web?

HOW MANY

How many times

one search, or several in a loop

…AND THE PATHS IT CAN CHOOSE

Skip retrieval	answer directly
Search DB	internal docs
Web search	current info
SQL query	structured data
Multiple sources	combine + synthesize

The agent treats retrieval as a tool it can use — not a mandatory step every query must pass through.

The Shift: Pipeline vs Agent

From a fixed track to a reasoning loop

CLASSIC RAG rigid pipeline

One fixed track. Every query is forced down the same three steps.

User Query

↓

Retrieve

↓

Generate

↓

Answer

Same path, every time — even when it makes no sense.

AGENTIC RAG reasoning loop

The agent reasons, picks a path, checks its work, and retries if needed.

User Query

↓

Agent reasons → chooses a path

skip | search DB | web | SQL | multi-source

↓

Evaluate quality

✓ good → use the results — Generate answer

✗ poor ↺ loop back & try a different path

Adapts the path to the actual question.

The agent is in control — not the pipeline.

retrieval is a decision, not a step

Query Classification Example

The analyzer prompt that routes every query

query_analyzer.py

```
QUERY_ANALYZER_PROMPT = """Analyze this query and decide the best approach:

Query: "{query}"

Classify as one of:
- DIRECT_ANSWER:  Can answer from general knowledge, no retrieval needed
- SINGLE_SOURCE:  Needs one specific document / database
- MULTI_SOURCE:   Needs info from multiple sources
- CLARIFICATION:  Query is too vague, need to ask user

Also specify: which tool(s) to use, what search query to formulate.

Output JSON:"""
```

Query Classification: Examples

KAUST Academy

Same classifier, four very different routes

ONE CLASSIFIER · FOUR ROUTES — the strategy follows directly from the classification

USER QUERY	CLASSIFIED AS	STRATEGY
"What's the capital of France?"	DIRECT_ANSWER	No retrieval — answer from general knowledge
"What's our refund policy?"	SINGLE_SOURCE	search_internal_docs("refund policy")
"Is our Berlin office affected by the new EU regulation?"	MULTI_SOURCE	web_search(...) + search_internal_docs(...)
"Tell me about the thing we discussed"	CLARIFICATION	Ask the user for specifics before searching

MEASURED Our first prompt called both two-document questions **SINGLE_SOURCE**. One rule saying when a question needs more than one document fixed it.

Retrieval as a Tool

The description is the routing logic

This is the **tool schema the model reads** to decide whether to retrieve — **not** the tool's implementation code. The agent only ever sees this description and these parameters.

tool_schema.json

WHAT THE MODEL SEES

```
{
 "name": "search_knowledge_base",
 "description":
  "Search the company knowledge base for policies, products,
   and processes. Do NOT use for general knowledge. Do NOT use
   for current events — use web_search instead.",
 "parameters": {
  "type": "object",
  "properties": {
   "query": {"type": "string", "description": "specific terms, not the full question"},
   "department": {"type": "string"},
   "top_k": {"type": "integer", "default": 5}
  },
  "required": ["query"]
 }
}
```

HOW THE AGENT READS IT

The description is the routing logic

The highlighted text is the only thing telling the agent **when** to reach for this tool — and when not to.

Parameters shape the call

They tell the model what to pass — specific terms, optional filters, a result count.

Implementation stays hidden

The actual DB query, embeddings, and ranking code live elsewhere. The model never sees them.

Write the description like instructions, not a label.

Day 3: vague vs specific descriptions, **4/12 vs 11/12.**

Multiple Retrieval Tools

Give the agent options — it picks, maybe several, maybe none

Tool	When to Use
search_docs	Internal policies, procedures, guides
web_search	Current events, external information
query_database	Structured data, numbers, metrics
search_code	Codebase, documentation, READMEs
No tool call → answer directly	General knowledge — no retrieval needed

Agent decides which to call. Maybe multiple. Maybe none.

Self-Correction: The Key Advantage

After retrieving, the agent grades its own results

After retrieving, the agent **grades its own results** against four checks before it dares to answer

Relevant? 1

Do these chunks actually answer the question?

E.G. *Asked about refunds, but got the shipping policy → not relevant.*

Complete? 2

Do I have enough, or do I need more?

E.G. *Found the US policy, but the question spans all regions → need more.*

Correct source? 3

Did I search the right place?

E.G. *Searched internal docs for a live stock price → should hit the web.*

Contradictions? 4

Do the sources disagree with each other?

E.G. *One doc says 30 days, another says 60 → resolve before answering.*

If any answer is "no" → retry with a different strategy.

1. Asai et al., Self-RAG, ICLR 2024 2. Yan et al., Corrective Retrieval Augmented Generation (CRAG), 2024

Self-Correction: Worked Example

Measured: asking about “Germany” passed on the first try; “Cologne”, which no document mentions, needed a retry

THE QUESTION

“How many unused PTO days can employees in our Cologne office keep?”

Agent searches — search ranks the **US policy** first, because “Cologne” appears in no document — ATTEMPT 1

Agent evaluates — *“US, not Germany.”* It rejects the result. — SELF-CHECK ✗

Agent adds a filter & retries — adds `country="Germany"` and searches again — ATTEMPT 2

Agent gets the right document — the **Germany-specific** HR policy — SELF-CHECK ✓

Agent generates the answer — accurate and grounded in the correct source — DONE

Classic RAG would have stopped at attempt 1: it would answer from the US policy, confidently wrong, or at best say “I don’t know.”

Re-Retrieval Strategies

KAUST Academy

When the first retrieval returns bad results

First retrieval returned bad results → escalate through these moves, in order:

Rephrase the query
Try more specific or different terms

E.G. "PTO" → "paid time off rollover limit"

Add metadata filters
Narrow by department, date, or country

E.G. country="Germany"

Try a different tool
Switch source entirely

E.G. web_search instead of internal docs

Decompose the query
Split into smaller sub-questions

E.G. "compare A vs B" → search A, then B

Give up gracefully
Be honest rather than hallucinate

E.G. "I couldn't find this information."

Max retries: 2–3. After that, tell the user honestly — don't keep burning tokens.

Multi-Step Retrieval

KAUST Academy

Chaining searches to answer what a single lookup can't

"Are we on track to hit this quarter's sales target?"

SINGLE-SHOT RAG — one search

```
search_docs("Q4 sales target")
```

answer

"The Q4 target is **$5M**."

❌ Repeats the goal — never says if we'll hit it.

MULTI-STEP RETRIEVAL — two sources + math

```
search_docs(target)
Target $5M

query_db(sales)
So far $3.2M
```

compare

"At **$3.2M** after 10 of 13 weeks, you're at **64%** of the target with **77%** of the quarter gone: **behind pace** to hit $5M."

MEASURED · CHECK THE ARITHMETIC

Our model got **$1.8M left** and **$600K a week** right but never computed the pace. Give it a calculator tool (Day 3).

✅ Answers the real question — with a verdict.

The effect: chaining searches turns a restated goal into a decision-ready verdict.

Agentic RAG: Full Architecture

KAUST Academy

Seven steps, with a retry loop in the middle

1. **Analyze query** — what do I need?
2. **Choose tool(s)** — which source?
3. **Formulate query** — what search terms?
4. **Execute retrieval** — run the search
5. **Evaluate results** — good enough?

NO — retry

Results fall short → **refine and loop back to step 2** with a new tool or new terms.

Cap: 2–3 attempts, then answer honestly.

YES ↓

6. **Generate answer** — with citations

BUILD IT WITH **Plain Python (today's labs)** **LangGraph** Frameworks compared on Day 5

PART 2

Multi-Agent Orchestration

When one agent isn't enough



The Single-Agent Ceiling

KAUST Academy

Pile on enough jobs and one agent buckles



ONE agent

CONTEXT WINDOW 110%

overflowing

✓	**Retrieve from 5 sources**	docs, web, SQL, email, code — all at once
✓	**Reason over complex data**	cross-reference figures, spot trends
✓	**Write & run code**	pull a report, then analyze the numbers
✓	**Generate polished output**	format a client-ready summary
✗	**Juggle 15+ tools**	which tool for which step? it starts guessing
✗	**…and everything else**	instructions pile up and contradict

Context window fills → instructions contradict → performance degrades. The fix isn't a bigger prompt — it's splitting the work across agents.

MEASURED Day 3: four tools added **~685 tokens** to every call. Fifteen tools add **several thousand**.

Single Agent vs Multi-Agent

More capable, but more to manage

Single Agent	Multi-Agent
one generalist doing it all	a team of focused specialists
CONTEXT Shared across everything	**CONTEXT** Each agent has its own
PROMPT One prompt, many rules	**PROMPT** Focused prompt per agent
TOOLS All loaded at once	**TOOLS** Each agent gets only the tools its job needs
DEBUGGING One trace to follow	**DEBUGGING** Multiple traces
COST Lower (fewer calls)	**COST** Higher (more calls)
QUALITY Degrades with complexity	**QUALITY** Holds up per-domain

Multi-agent buys capability and quality — at the cost of more calls, more spend, and harder debugging. Split only when the payoff is worth it.

When to Go Multi-Agent

A decision framework — don’t over-engineer

KAUST Academy

START HERE

Can one agent handle it well?

YES → Stay single-agent

Simpler, cheaper, easier to debug. Don’t over-engineer.

GO MULTI-AGENT ONLY WHEN …

More than ~10 tools	→ Split by domain
Needs different expertise	→ Specialist workers
Tasks run independently	→ Parallel agents
Needs iterative refinement	→ Peer-review loop
Spans multiple data sources	→ Multi-agent RAG

Default to a single agent. Reach for multi-agent only when a concrete signal forces the split.[1]

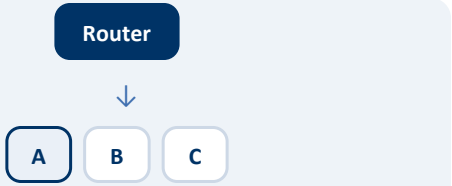
1. Anthropic, “Building Effective Agents,” Dec 2024: find the simplest solution first; add complexity only when it pays off

The Three Orchestration Patterns

Router · Supervisor · Peer

1 Router

the traffic cop



Classify the query, then send it to exactly one agent.

EXAMPLE

"Reset my password" → IT agent. Nothing else runs.

2 Supervisor

the manager



Break the task into subtasks, delegate to workers, combine results.

EXAMPLE

"Q3 report" → data-pull + analysis + writing.

3 Peer

the team



Agents hand work to each other directly, with no central authority.

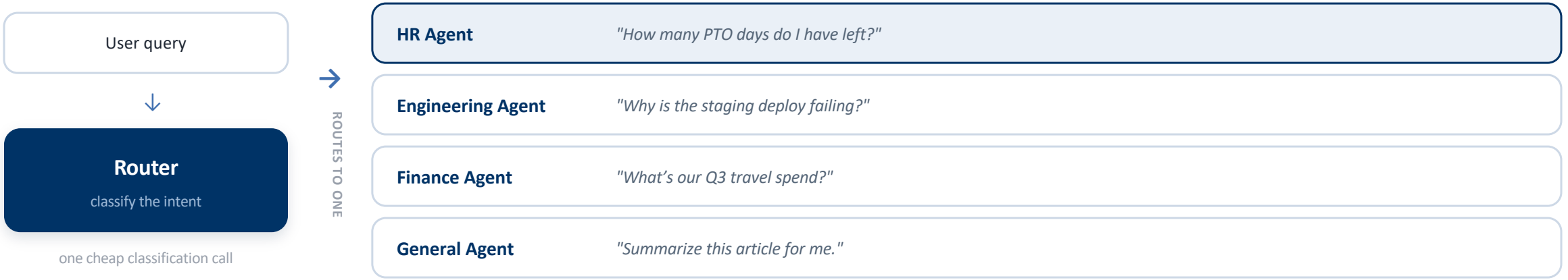
EXAMPLE

Writer ↔ Editor loop until the draft is approved.

These aren't mutually exclusive — production systems layer them: a router out front, a supervisor behind it, peer loops inside a worker.

Pattern 1: Router-Based Dispatch

Classify → send to the single best agent



No planning
the router never decomposes — it only picks

One agent runs
exactly one specialist handles the whole query

Best for
cleanly separable domains with little overlap

Router: Implementation

Often under 20 lines

router.py

```
ROUTER_PROMPT = """Classify the user query into exactly one category:
- HR:          hiring, PTO, benefits, org structure, employee relations
- ENGINEERING: code, architecture, deployments, bugs, infrastructure
- FINANCE:     revenue, costs, budgets, invoices, forecasting
- GENERAL:     anything that doesn't fit above

Query: {query}
Respond with only the category name."""

category = llm.invoke(ROUTER_PROMPT.format(query=user_query)).content.strip().strip(".").upper()   # message → text; "Finance." → FINANCE
response = agents.get(category, agents["GENERAL"]).run(user_query)   # unknown label → GENERAL
```

A router is often a small LLM. After collecting some data, a classifier can be trained for the task.

Router: Speed Optimization

Start with an LLM router, then distill (timings are illustrative)

LLM router ~500 ms

Most flexible, slowest. Start here to learn the traffic.

EXAMPLE

A small-model call classifies every incoming query.

Small classifier ~50 ms

Train once on logged data. 10× faster.

EXAMPLE

A fine-tuned classifier trained on past routes.

Rule-based <1 ms

Keyword match. Near-instant for the obvious cases.

EXAMPLE

"deploy" → ENGINEERING

router_fast.py

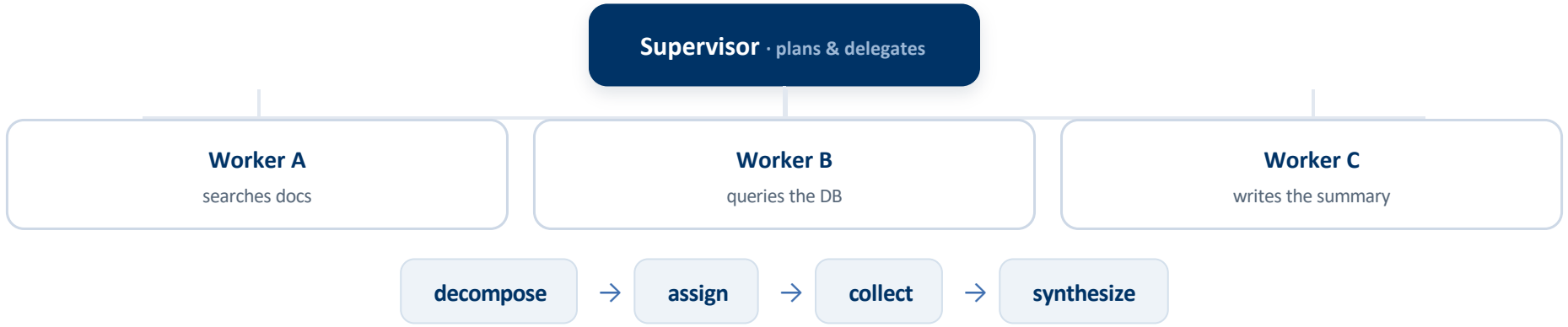
```
def route(query):                    # rules first, LLM as fallback
    if any(k in query.lower() for k in ["pto","hiring"]):
        return "HR"
    if any(k in query.lower() for k in ["deploy","bug"]):
        return "ENGINEERING"
    return llm_classify(query)       # only the ambiguous ones
```

Cheap rules catch the obvious queries first; fall back to the LLM only for the genuinely ambiguous ones.

Pattern 2: Supervisor-Worker

KAUST Academy

The supervisor manages — it doesn't do the work



WORKED EXAMPLE — "Draft a Q3 board update" · A and B run in parallel and report only to the supervisor

Supervisor
splits into 2 independent subtasks

→ **Worker A**
pulls Q3 KPIs from docs

→ **Worker B**
gets revenue from the DB

→ **Supervisor**
collects both results, merges into one update

The supervisor never does the work itself — it decomposes, delegates to specialists, and combines their results.

Supervisor: The Planning Step

Decompose into a structured execution plan

DAY 3 CALLBACK This is **Plan-and-Execute**, with agents as the executors.

supervisor.py

```
SUPERVISOR_PROMPT = """You are a supervisor managing specialist agents.

Available workers:
- HR Agent:         HR policies, benefits, employees
- Finance Agent:    budgets, costs, revenue
- Engineering Agent: technical questions about code and systems

User query: "{query}"

Create an execution plan as JSON:
{
  "subtasks": [
    {"worker": "HR Agent",      "task": "specific task description"},
    {"worker": "Finance Agent", "task": "specific task description"}
  ],
  "execution": "parallel" or "sequential",
  "synthesis_strategy": "how to combine results"
}"""
```

Supervisor: Worked Example

A travel assistant planning a 3-day business trip to Tokyo

USER "Plan my 3-day business trip to Tokyo next month."

Supervisor decomposes → runs 3 workers in parallel

Flights agent

searches airline APIs for the best round-trip

RETURNS

ANA, Tue–Fri, $980

Hotel agent

finds lodging near the meeting venue

RETURNS

Shinjuku, 4★, $180/night

Calendar agent

checks the user's schedule & timezone

RETURNS

Free Tue–Fri; JST +9

SUPERVISOR PROPOSES

"Found: ANA Tue–Fri ($980) and a Shinjuku hotel, 3 nights ($540). Fits your calendar (JST). Total **$1,520**, under your travel budget. **Book all three?"**

Approve | Edit

Worker Design Principles

KAUST Academy

Narrow, self-contained, constrained, structured

Narrow scope

Each worker does ONE job — not "I help with stuff."

EXAMPLE

"I search financial documents." ✓ "I'm a helpful assistant." ✗

Own system prompt

Its own specialty, constraints, and output format. Workers can't see each other's prompts.

EXAMPLE

Finance worker's prompt never mentions HR policy or code.

Restricted tools

Only the tools its job needs — nothing more.

EXAMPLE

HR worker: search_hr_docs only. No run_code.

Structured output

Return a JSON schema, not freeform prose, so synthesis is predictable. gaps names what the worker couldn't find.

EXAMPLE

{answer, sources, gaps}, with no self-reported confidence

Narrow, constrained workers are what make the supervisor's job predictable.

Supervisor: Synthesis Strategies

KAUST Academy

Three ways to combine worker outputs

1 Concatenation

when outputs don't overlap

Just join the results in order. Cheapest — no extra LLM call.

EXAMPLE

Flights + Hotel + Calendar → stitched into one itinerary.

2 Merge & deduplicate

when outputs overlap or conflict

Combine, drop duplicates, reconcile disagreements. One extra LLM call.

EXAMPLE

Two workers both cite revenue → keep one, flag any mismatch.

3 Structured report

when the user wants a document

Use the outputs as raw material for a formatted, sectioned write-up.

EXAMPLE

Worker findings → an exec summary with headed sections.

synthesis_prompt.py — the merge strategy

```
SYNTHESIS_PROMPT = """The user asked: {question}     # the fix
You received outputs from specialist agents:
  Flights agent: {flights}   Hotel agent:   {hotel}
Combine into ONE answer. Resolve any conflicts by noting both,
and cite which agent supplied which fact."""
```

MEASURED · GIVE THE SYNTHESIZER THE QUESTION

Without it, ours added two teams' hiring totals **($33,600 + $19,500)** instead of comparing them. With it, it compared them, but still missed that Finance costs more per hire **($6,500 vs $4,200)**.

Pick by how much the outputs overlap — concatenate independent results, merge overlapping ones.

Supervisor: Handling Worker Failures

KAUST Academy

What to do when a worker returns bad results

WORKER OUTPUT	SUPERVISOR RESPONSE	
Valid & complete	Accept it and move on.	EXAMPLE: Finance returns clean Q3 numbers → use them.
Partial	Ask the worker to continue, or fill the gap with another worker.	EXAMPLE: Got headcount but not spend → re-ask for spend.
Irrelevant	Rephrase the subtask and resend to the same worker.	EXAMPLE: Returned US data for a Germany query → add filter, retry.
Error / timeout	Retry once, then skip with an explanation.	EXAMPLE: API times out → one retry, else note it in the answer.
Contradicts another	Note both; supervisor decides or asks the user.	EXAMPLE: HR says 30 days, Finance says 60 → surface both.

Cap retries at 2 per worker and enforce a hard total budget per query — never let one worker stall the whole task.

Pattern 3: Peer Agents

Equals that collaborate directly — no central coordinator

Peers are **equals with no central coordinator** . Each agent does its part, then decides which peer to hand the work to next — collaborating and iterating directly.



THE RISK **With no coordinator, peers can loop forever. Cap the number of revision rounds.**

No hierarchy

Nobody is the boss. Agents negotiate as equals.

Direct handoffs

Each agent returns a next field naming the peer who goes next.

Best for

Iterative work: draft → review → revise.

Peer Agents: Implementation

Structured output makes the "next" decision reliable

peer_agents.py

```
# Give the editor a SCHEMA so its output is parseable —
# llm.invoke() alone returns text, not an .approved field.
class Review(BaseModel):
    approved: bool
    feedback: str

def editor_agent(state):
    review = llm.with_structured_output(Review).invoke(
        f"Review this draft:\n{state['draft']}")
    if review.approved:                # now a real bool
        return {"final": state["draft"], "next": "END"}
    return {"editor_feedback": review.feedback,
            "next": "writer", "stage": "revision"}
```

Any time you branch on `review.approved`, bind a schema with `with_structured_output()` — never parse raw LLM text by hand.

Peer vs Supervisor: Trade-offs

Centralized control vs distributed flexibility

Aspect	Supervisor	Peer
Control	Centralized, predictable	Distributed, flexible
Bottleneck	Single point of failure	No single point of failure
Debugging	Easy — follow supervisor log	Hard — trace agent-to-agent msgs
Best for	Clear decomposition tasks	Iterative refinement workflows
Main risk	Misrouting kills the pipeline	Infinite loops between agents
Scaling	Add workers easily	More agents = more comms overhead

Supervisor for clear decomposition; peer for open-ended iteration — production systems often nest a peer loop inside a supervisor.

Hybrid: Layering Patterns

KAUST Academy

Production systems combine all three

A **hybrid** system doesn't pick one pattern — it **stacks all three as layers**. A query enters at the top and only descends to the next layer if it needs more coordination.

QUERY "Research our top 3 competitors and write me a one-page brief."

↓

1	**Router** first line	IN OUR EXAMPLE Not a simple lookup → it's a multi-step research job → escalates down.
2	**Supervisor** the planner	IN OUR EXAMPLE Spawns one **Research agent per competitor**, all running in parallel.
3	**Peer loop** inside a task	IN OUR EXAMPLE A **Writer ↔ Editor** loop drafts and tightens the brief until it's clean.

↓

RESULT **Supervisor stitches the 3 research findings into one polished one-page brief.**

RULE **One agent can answer → route. Several needed → plan.**

Sequential vs Parallel Execution

And the DAG hybrid most production systems use

The execution shape follows the **data dependencies** — run steps back-to-back only when one needs the previous one's output.

Sequential

one after another



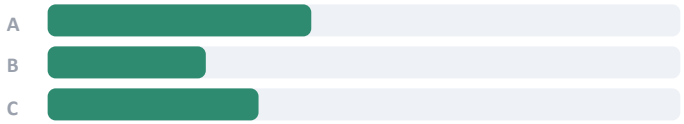
Total = A + B + C = 12

EXAMPLE

Research → analyze → write: each needs the last one's output.

Parallel

all at once



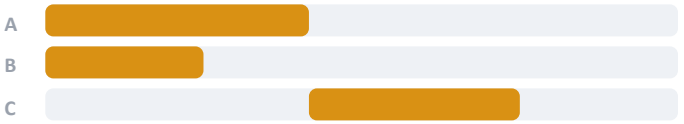
Total = max(A,B,C) = 5

EXAMPLE

Search 3 competitors' sites at once — none depends on another.

Hybrid (DAG)

parallel, then join



Total = max(A,B) + C = 9

EXAMPLE

Pull 3 data sources in parallel, then analyze together.

Parallelize independent work — it cuts wall-clock time from the *sum* of tasks to the *slowest* one.

MEASURED **Parallel isn't free: same tokens. On one GPU it means batching;** batching 3 independent calls ran **3.1×** faster than one by one (our test, CPU).

DAG Example: Hybrid Execution

"How does our engineering hiring compare to competitors?"

WHAT A DAG IS

A **directed acyclic graph**: each step is a node, an arrow means “needs the result of”, and no arrow loops back. Steps with no arrow between them **run at the same time**.

PARALLEL PHASE · 5 min (the slowest of 3, 5, 2)

- **HR Agent** · our hiring data (3 min)
- **Web Agent** · competitor data (5 min, the slowest)
- **Finance Agent** · our hiring budget (2 min)

→

SEQUENTIAL PHASE · 5 min (3 + 2, after all three finish)

- **Analysis Agent** · compare all three (3 min)
- **Writer Agent** · format exec summary (2 min)

DAG: 5 + 5 = 10 min

Pure sequential: 3 + 5 + 2 + 3 + 2 = 15 min

WHY 10, NOT 15

The total is the longest chain of steps that wait on each other: Web → Analysis → Writer = 5 + 3 + 2 = 10 min. Making HR or Finance faster saves nothing.

When Planning Hurts Performance

KAUST Academy

Coordination overhead can exceed the benefit

Multi-agent isn’t free — planning, delegation and synthesis all cost tokens, latency and failure surface. When the task is small or ill-posed, that overhead **outweighs any benefit** .

Simple queries

Decomposing a 2-second question into plan + workers + synthesis. The coordination overhead dwarfs the work itself.

Symptom: a one-line answer takes seconds and several LLM calls to produce.

COST **2× the calls, 2.3× the time (our test)**

EXAMPLE

"What’s our office address?" → one direct answer beats spinning up 3 agents.

Bad plans

A misread query yields the wrong subtasks. Every worker then does wasted work — and the error is multiplied across all of them.

Symptom: confident, well-formatted output that answers the wrong question.

COST **wasted spend × N workers**

EXAMPLE

Supervisor mis-splits the ask, so all 3 workers solve the wrong problem in parallel.

Vague scope

With no clear success criteria the planner keeps decomposing and re-delegating, never converging on "done".

Symptom: agents churn in circles until a limit or budget cap trips.

COST **loops until the guardrail fires**

EXAMPLE

"Improve our docs" → endless re-planning with no definition of finished.

RULE OF THUMB **Route simple queries directly. Reach for multi-agent only when the task genuinely needs several specialists working together.**

PART 3

Agent Communication & State

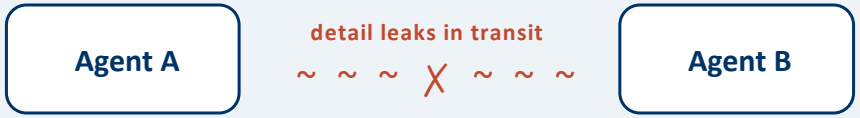
How agents share information without losing context

The Communication Problem

KAUST Academy

When Agent A needs to tell Agent B something

When one agent hands off to another, the message is the only thing that survives. Three choices decide whether the receiver gets what it needs — or a game of **broken telephone** .



1 · What format?

Free text is easy to write but hard to parse. Structured JSON or a typed schema keeps hand-offs machine-readable.

EXAMPLE

A worker returns {answer, sources, confidence} — not a paragraph.

2 · How much context?

Send too little and B is guessing; send the whole transcript and you burn tokens and bury the point.

EXAMPLE

Pass the sub-task *plus* the original question — not 40 turns of chat history.

3 · Where does it live?

Direct messages are simple but fragile; a shared store or queue scales to many agents.

EXAMPLE

LangGraph keeps one TeamState every node reads and writes.

WHY IT MATTERS **Get any of these wrong → agents act on incomplete information → confidently wrong outputs.**

Three Communication Patterns

Message passing · Shared state · Blackboard

KAUST Academy

Agents need a channel to share information. The three patterns trade **simplicity** against **how well they scale** as the number of agents grows.

SIMPLEST — SCALES TO MANY AGENTS

Message passing

Agent A sends its output straight to Agent B — like email.

WHO RUNS NEXT **the sender: A calls B**

How — each agent calls the next with its result as input.

Trade-off — simplest to build; context is easily lost across hops.

Best for — 2–3 agents in a fixed sequence.

EXAMPLE

Router hands the query text directly to the chosen worker.

Shared state store

A central record every agent reads from and writes to.

WHO RUNS NEXT **the supervisor or graph, in code**

How — agents read and write fixed, named fields.

Trade-off — scales well, stays traceable; needs concurrency care.

Best for — supervisor + many workers.

EXAMPLE

Every worker writes into one TeamState dict (LangGraph).

Blackboard

A shared workspace agents post findings to and pull from.

WHO RUNS NEXT **each agent, when it sees work**

How — agents post loose findings and watch the board.

Trade-off — very flexible, decoupled; harder to trace and debug.

Best for — open-ended, emergent collaboration.

EXAMPLE

A research agent posts facts; a writer picks them up when ready.

RULE OF THUMB **Two or three agents? Message passing. More? A shared state store. A blackboard only when you can’t plan the order up front.**

Shared State: LangGraph

Every node reads from state and writes to state

team_state.py

```python
from langgraph.graph import StateGraph
from typing import Annotated, TypedDict
import operator

class TeamState(TypedDict):
    query: str              # Original user question, never modified
    plan: list              # Supervisor's plan
    research_findings: str  # Research worker output
    financial_data: str     # Finance worker output
    analysis: str           # Analysis worker output
    final_answer: str       # Synthesized answer
    messages: Annotated[list, operator.add]  # Reducer: parallel writers append
    errors: list            # Failures and retries, for recovery
    approval_status: str    # Human approvals still pending
    metadata: dict          # Tokens, costs, timings

graph = StateGraph(TeamState)
# Each node reads from state and writes to state — no direct messages needed
```

PARALLEL WRITES

Two parallel nodes writing one key need a reducer: **Annotated[list, operator.add]**.

WHY IT WORKS **No agent messages another directly — the typed state IS the channel, so every node stays in sync and the run is fully traceable.**

Inter-Agent Message Format

Structured beats freeform

message.py

```
# Don't: freeform text between agents
message = "I found the revenue was $5.2M but I'm not sure about Q4"

# Do: structured messages
message = {
    "from": "finance_agent",
    "to":   "supervisor",
    "type": "task_result",
    "content": {
        "answer":     "Q3 revenue was $5.2M",
        "sources":    ["financial_report_q3_2024.pdf", "page 12"],
        "gaps":       ["Q4 data not yet available"],
        "tokens_used": 1450,
        "latency_ms":  2300
    }
}
```

Structured = parseable, debuggable, auditable.

> **TAKEAWAY** **Structured messages are parseable, debuggable and auditable — freeform text is none of these.**

Memory in Multi-Agent Systems

Local · Shared · Long-term

KAUST Academy

Multi-agent systems keep memory at three scopes. The wider the scope, the longer it lives — and the more deliberately you must decide what to write.

NARROW & SHORT-LIVED — WIDE & PERSISTENT

Local · per agent

A private scratchpad the agent uses to reason through its own task.

Scope — one agent only

Lifetime — ends when the agent finishes

Stored in — the agent's context window

EXAMPLE

A worker's chain-of-thought while solving one subtask.

Shared · across agents

The team's working memory that every agent reads from and writes to.

Scope — the whole team

Lifetime — the current task/run

Stored in — the shared state store

EXAMPLE

The supervisor's plan + each worker's output in TeamState.

Long-term · across sessions

Knowledge that outlives a single run so the system improves over time.

Scope — user / organisation

Lifetime — persists across runs

Stored in — a vector DB or key-value store

EXAMPLE

Recalling a user's past tickets and preferences next session.

RULE OF THUMB **Keep working data in shared state; promote only durable, reusable facts to long-term memory.**

PART 5

Failure Modes & Reliability

Everything that goes wrong — and how to survive it



The Top 5 Delegation Failures

KAUST Academy

The classic ways multi-agent systems break

MAST catalogs **14 failure modes in 3 groups**[1]. These five are the delegation failures we hit in our own runs.

			MEASURED IN OUR RUNS
1	**Misrouting**	Supervisor sends a task to the wrong worker.	*First triage prompt routed 1 of 5 correctly*
2	**Lost context**	Key detail dropped when passing between agents.	*“Berlin” kept; “started after Jan 2023” lost*
3	**Infinite loops**	A delegates to B, B delegates back to A, forever.	*Researcher ↔ supervisor ping-pong*
4	**Conflicting outputs**	Two workers return contradictory answers.	*Two critics disagreed on 2 of 5 replies*
5	**Over-delegation**	A 2-second question split into 5 subtasks.	*Team used 16 calls; one agent used 8*

1. Cemri et al., “Why Do Multi-Agent LLM Systems Fail?” (MAST), NeurIPS 2025; arXiv 2503.13657

Infinite Loop Detection

KAUST Academy

Three guardrails: steps, cost, repetition

Agents can get stuck — delegating back and forth, retrying the same failing step, or circling a task with no exit. A **loop detector** wraps the run and trips on any of three independent limits.

1 · Step ceiling

Trips at: max iterations

Cap how many reasoning/delegation steps a single run may take. A hard stop that always fires eventually.

CATCHES

An agent that keeps "thinking" without ever acting.

2 · Cost budget

Trips at: max $ / query

Track spend across every model call. When the run exceeds its budget, stop and report — never surprise-bill.

CATCHES

A branching plan that quietly spawns dozens of calls.

3 · Repetition

Trips at: repeated state

Hash the state each step; if the same state recurs, the system is going in circles — halt it.

CATCHES

Two agents handing the identical task back and forth.

A PING-PONG LOOP, CAUGHT

1. Supervisor → Researcher: "get the figure"
2. Researcher → Supervisor: "need more detail"
3. Supervisor → Researcher: "get the figure"
4. ! **Same state seen twice → LoopDetected, run halts**

MEASURED · HASH A SHORT STATE

In our ping-pong the round-3 reply wasn't a word-for-word copy, so a **raw-text hash** missed the repeat; **(document read, kind of reply)** caught it.

Without a detector this repeats until it burns the token budget — or forever.

SET ALL THREE **Steps and cost bound the worst case; repetition catches the silent loops. On any trip, fail gracefully and hand off to a human.**

Mitigation Strategy Checklist

KAUST Academy

Eight guardrails for production multi-agent systems

Most multi-agent failures are cheap to prevent and expensive to debug. These eight guardrails fall into three jobs — **bound the run, design it cleanly, and fail safely**.

BOUND THE RUN

✓ Max iteration limits

Hard-cap the reasoning/delegation steps a run may take.

`max_steps = 5 per worker`

✓ Cost budgets

Stop and report once a query exceeds its spend cap.

`budget = $0.10 / query`

✓ Timeouts

Don’t let one slow worker stall the whole pipeline.

`timeout = 30s per call`

DESIGN IT CLEANLY

✓ Clear agent descriptions

Unambiguous scope with no overlap, so routing is unambiguous.

`one job per agent`

✓ Structured output schemas

Typed results in, typed results out — less garbage to synthesize.

`return JSON schema`

✓ Shared state + versioning

One source of truth that records who wrote what, when.

`state.v, writer tags`

FAIL SAFELY

✓ Fallback to human

When a check fails, escalate with context instead of guessing.

`check failed or retries used up → handoff`

✓ Adversarial testing

Feed ambiguous and trick queries before real users do.

`red-team the router`

SHIP CHECKLIST **Add the limits first — they turn "runs forever / bills forever" into a bounded, debuggable failure you can recover from.**

Debugging Multi-Agent Systems

KAUST Academy

The trace is everything

Debugging many agents is debugging a **chain**. The failure you see is rarely where the error began — so work the trace backward to its source.

1. **Check the trace**
   Read the run end-to-end: which agent did what, in what order, with what inputs.

2. **Find the first bad step**
   Errors cascade — fix the earliest wrong output, not the loudest symptom.

3. **Isolate the component**
   Classify it: a tool bug, an agent (prompt) bug, or a coordination/hand-off bug?

4. **Replay with logging**
   Re-run the same query with verbose logging to reproduce it deterministically.

5. **Test the component alone**
   Pull the failing agent out and test it in isolation to confirm the fix.

READING A BROKEN TRACE

router	classified as BILLING ✓
supervisor	split into 2 subtasks ✓
finance ✗	returned Q2 figure, not Q3
analysis	built on the wrong figure
supervisor	synthesized a confident, wrong answer

FIRST BAD STEP

The finance agent, not the analysis — fix its retrieval query, not the summary.

THE RULE **The trace is everything. If you can’t trace who did what and why, you can’t debug it — instrument first, debug second. Day 5: tracing tools.**

DAY 4

Thank You

Multi-Agent Systems & Human-in-the-Loop — from single agents to coordinated, safe AI systems.

YA

CONTRIBUTED BY

Yazan Alshuaibi